CSC-239
Java Programming: Introductory Lecture

September 3, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

CSC-239: Java Programming

- Instructor: Peter Morgan
- Email Address: pmorgan@bhcc.edu
- Section Number: CSC-239-02
- Class Meetings: remotely via Webex.

  Mon, Wed: 9:00 a.m. – 10:40 a.m,
  9/3/2025 – 12/15/2025

- Remote access to class sessions:

  https://bhcc.webex.com/meet/pmorgan

  Connect to the class using the *Webex Meetings* application, running on your personal computer.

- Office Hours: after class or by appointment.

Procedures of the Course

- Communication outside of class will primarily be using BHCC e-mail.

  It is important to check your BHCC e-mail account frequently.
- Class materials will be distributed via *Moodle*.
- Programming Projects and Lab Exercises will be submitted via *Moodle*.

  Students must also **demonstrate** the solutions to their Lab Exercises to the instructor during class.
- This lecture is being recorded.

  I will post a link to the recording on *Moodle*, when it is available.

Most Important Pre-requisite: CSC-120

IMPORTANT: CSC-239 is *not* intended as an "introduction to programming" course.

- If you have not met the prerequisites, particularly CSC-120, then you may find this course very difficult and fast-paced.
- Other programming courses or prior experience *may* be a suitable substitute for CSC-120 , but please talk to me about this after class.

Pre-requisite Skills

- Understand how to use basic programming statements in *some* programming language, such as C or C++:
  - variables of different types
  - assignment statements, expressions
  - decision statements
    - `if-else` statements
    - `switch` statements
  - loops
    - `for` loops
    - `while` loops
    - `do-while` loops
  - functions
- Understand the "step-by-step" nature of programming.

Syllabus

- The Syllabus is the primary planning document for any course.
- The Syllabus states the policies by which the course will be conducted.
- Our first Quiz will include questions about the contents of the Syllabus.

Announcements

- Quiz 1 will begin on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will open Wednesday evening, at 9:00 p.m.
  - The Quiz will remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

A Process for Learning Java

1. Read as much of the assigned chapter as you can readily understand.
2. Experiment with the sample code from the textbook, if this helps you understand.
3. Use the online documentation as a reference, for:
   - Checking details you may have forgotten.
   - Answering particular questions you may have.
4. Do not let the *volume* of online information overwhelm you. Java is a vast subject, which takes time and gradual exposure to learn.
5. **Ask questions** (in class, and by e-mail), *even if you are behind in the reading*. (**If** I need to answer a particular question privately, then I will tell you that.)
6. Repeat these steps as you incrementally work through the reading and programming assignments.

Characteristics of Java

- Many syntax similarities to other languages, such as C and C++, but not identical.
- Standard class libraries are different from other languages.
- “Compile once, run anywhere.”
- Object Oriented (more on what that means later)

A Very Simple Java Program

```
public class Welcome{
  public static void main(String[] args) {
    // Display welcome message on console.
    System.out.println("Welcome to Java!");
  }
}
```

- This program outputs a "Welcome to Java!" message to the console.
- "`System.out`" is the object that corresponds to the standard output device (the console).

Mechanics of Java Development

- Two basic types of user interface:
  - Command-line
  - Integrated Development Environment (IDE)
- Command-line Java Development
  - First, you would need to install the **Java Development Kit** (JDK). (That is a multi-step process that we will not discuss today.)
  - Edit your program source file using some text editor (for example, `notepad`).
  - Compile the program:

```
javac Welcome.java
```

  - Run the program:

```
java Welcome
```

The **`javac`** Compiler

What happens when you type the **`javac`** command?

- The compiler program reads the **`.java`** source file and translates the code into one or more **`.class`** files.
- If the code uses Java classes not defined in the current file, the compiler will look for the required **`.class`** files, or **`.java`** files, compiling them as needed.
- If errors are detected then **`javac`** outputs messages which (hopefully) help you fix the problems.

The `java` Virtual Machine Program

What happens when you type the **`java`** command?

- The Java Virtual Machine (JVM) program reads the **`.class`** file(s) and attempts to execute the code.
- If the JVM encounters a run-time error, it will hopefully output a "runtime exception" message, and may or may not halt execution.

Integrated Development Environment

- Do we really need to put up with this "command-line" interface?
  - Sometimes "yes", sometimes "no".
  - Usually, we can use a software program called an Integrated Development Environment (IDE), which provides a graphical user interface for editing, compiling and running our programs.
- Our first lab exercise is an introduction to the mechanics of using the **GreaterThan** IDE to write a Java program.

Alternative IDE Choices

- Chapter 1 of our textbook includes brief discussions of two popular IDEs:
  - Apache Netbeans
  - Eclipse
- Different people have different preferences of an IDE.
- Personally, I have done most of my Java programming using the Netbeans IDE.
- For most of the work in this course, you can use the IDE of your choice, because what you submit for your assignments will be ONLY the source code file(s).

OO Programming Concepts

- Java uses Object-Oriented Programming (OOP) terminology from the very beginning, but our textbook does not really explain it until Chapter 9!
- A reasonable discussion of OOP Concepts can be found at:

https://docs.oracle.com/javase/tutorial/java/concepts/index.html

(Portions of the next several slides are taken from this section of the Java online Tutorial, and are therefore Copyrighted by Oracle Corporation.)

Objects have *state* and *behavior*

- Real-world objects are all around us:
  - Ceiling Fan
  - Retractable Ballpoint Pen
- Real-world objects have *state*:
  - A fan is "on" or "off".
  - If the fan is "on", it is running at a particular "speed".
- Real-world objects have *behavior*:
  - The normal way we (users) change the state of the fan is to use the switch: turn the fan "on/off" and "set the speed".

Another Real-World Example

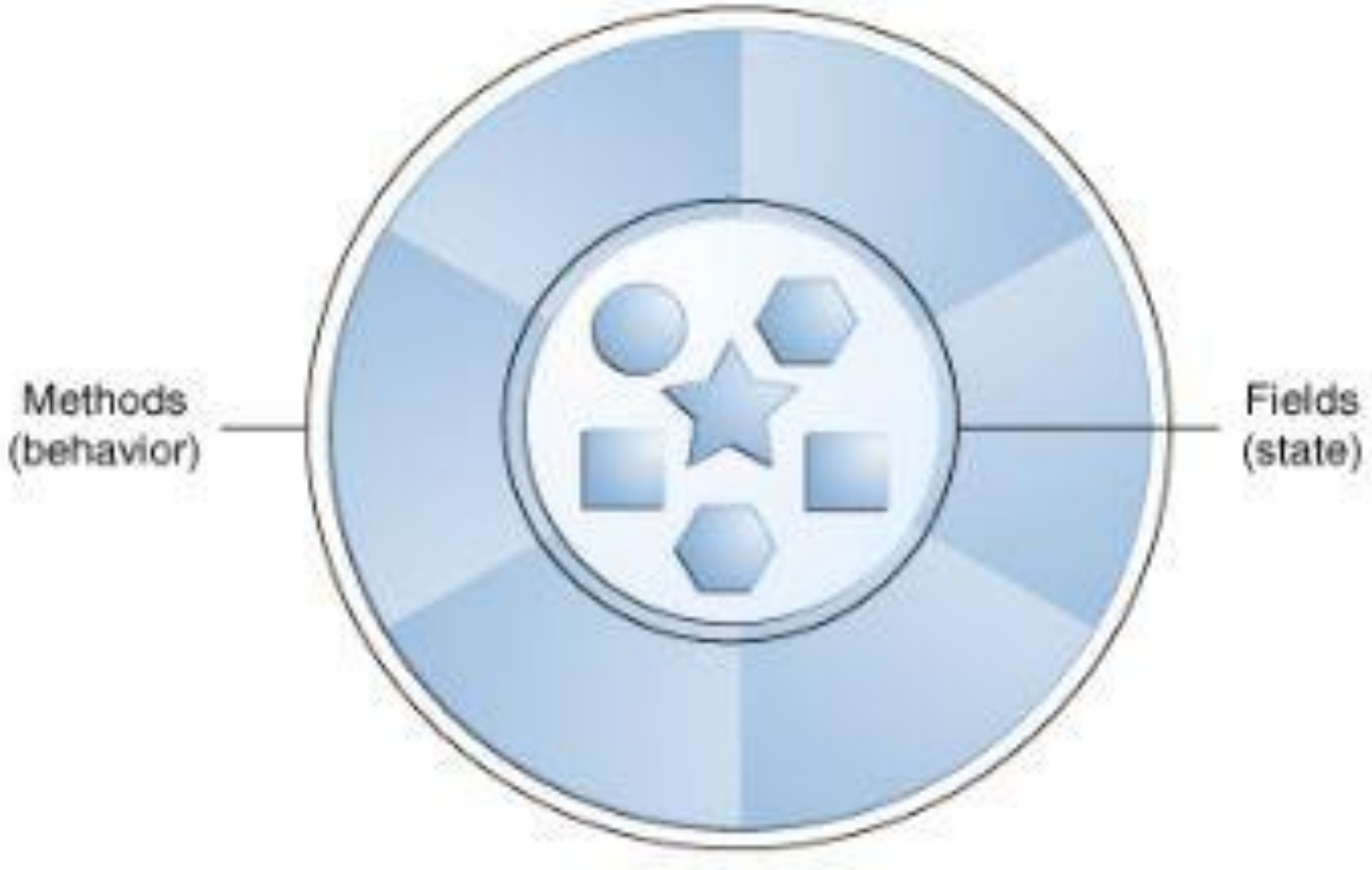
- A Retractable Ballpoint Pen has *state*:
  - Tip is extended or retracted.
  - The ink in the pen has a pre-determined color.
  - Some amount of ink remains in the pen.
- A Retractable Ballpoint Pen has *behavior*:
  - Extend or retract the tip (push the button)
- The normal way we change the state of the object is to push the button.

Software Objects

- Often we create software "objects" which correspond to real world objects:
  - A college course
  - A student
  - An instructor
- Software objects have *state* (fields):
  - student ID number
  - student GPA
  - current course(s)
- Software objects have *behavior* (methods):
  - add / drop a course
  - calculate GPA

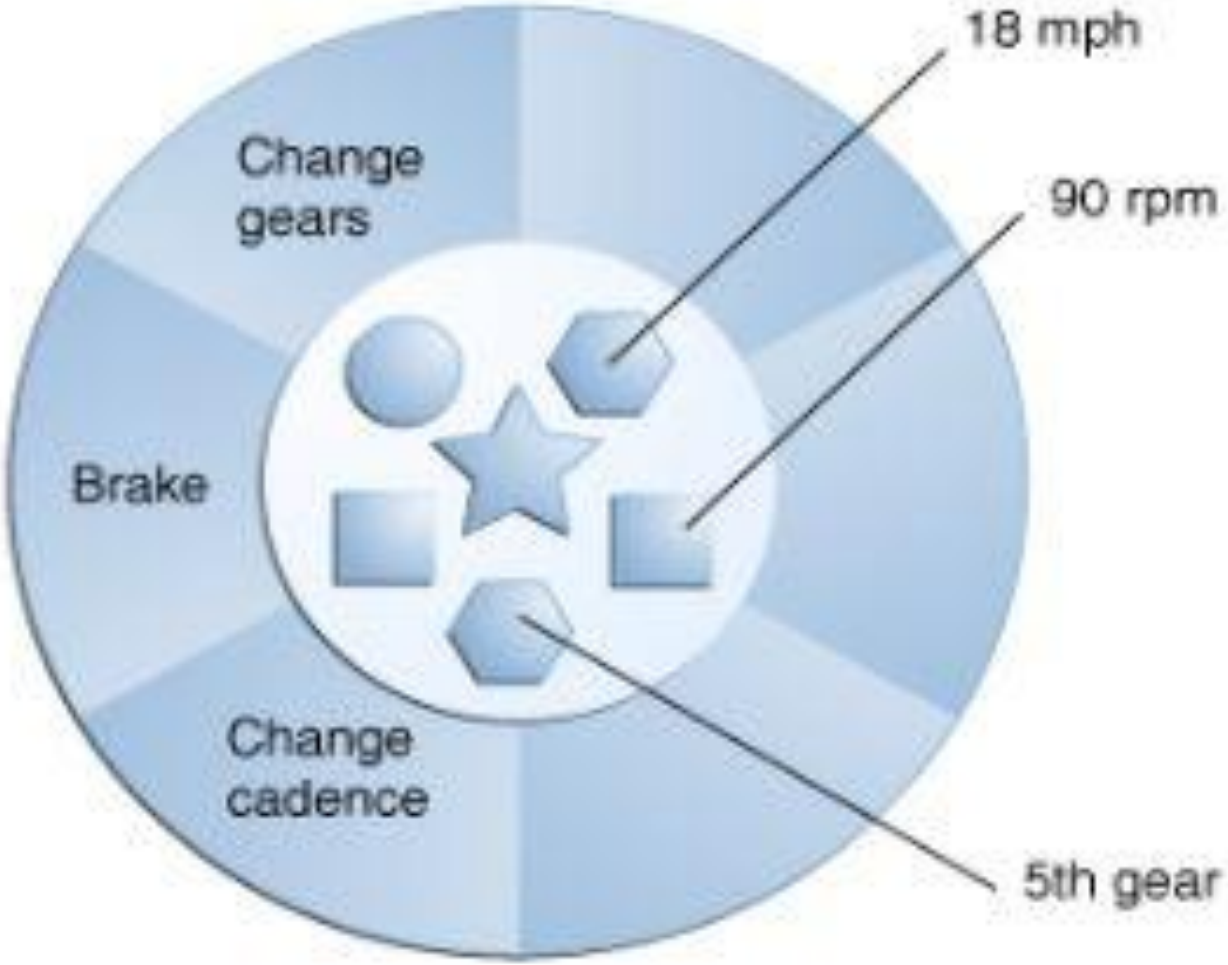
Internal State is "hidden"

- The Java Tutorial uses the following diagram to illustrate "*data encapsulation*":



Example from the Tutorial

- A bicycle modeled as a software object:



Bicycle Class (1)

```
/*
 * Copyright (c) 1995, 2008,
 * Oracle and/or its affiliates.
 * All rights reserved.
 */
class Bicycle {
            // Member variables (state):
    int cadence = 0;
    int speed = 0;
    int gear = 1;
```

Bicycle Class (2)

```
        // Methods (behavior):
void changeCadence(int newValue) {
        cadence = newValue;
 }

 void changeGear(int newValue) {
        gear = newValue;
 }

 void speedUp(int increment) {
        speed = speed + increment;
 }
```

Bicycle Class (3)

```
        // More Methods (behavior):
    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
} // (end class Bicycle)
```

BicycleDemo Class (1)

```
/*
 * Copyright (c) 1995, 2008,
 * Oracle and/or its affiliates.
 * All rights reserved.
 */
class BicycleDemo {
    public static void main(String[] args) {

        // Create two different
        // Bicycle objects
        Bicycle bike1 = new Bicycle();
        Bicycle bike2 = new Bicycle();
```

BicycleDemo Class (2)

```
// Invoke methods on object bike1:

bike1.changeCadence(50);
bike1.speedUp(10);
bike1.changeGear(2);
bike1.printStates();
```

BicycleDemo Class (3)

```
        // Invoke methods on object bike2:

        bike2.changeCadence(50);
        bike2.speedUp(10);
        bike2.changeGear(2);
        bike2.changeCadence(40);
        bike2.speedUp(10);
        bike2.changeGear(3);
        bike2.printStates();

    }   // (end main)
}   // (end class BicycleDemo)
```

Benefits of Using Objects

- Modularity: Logical separation of code sections, based on their purpose.
- Information-hiding: If the private internal details of the program change, the "outside world" does not need to change.
- Code re-use: A well written object can easily be shared, either within an organization, or beyond.
- Pluggability and debugging ease: If a particular implementation (of some library) has bugs, it is easy to supplement a different implementation as long as the interface is unchanged.

Java Online Documentation

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- Searching Java Online Documentation:

  Click the "Search" link (upper-right corner), enter your search text, and choose the most relevant link displayed.

Runtime Errors

- Some invalid operations may *compile* with no errors, but fail at execution time.
- For example, if you attempt to divide an integer value by zero, an "exception" is generated:

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
at FirstApp.main(FirstApp.java:25)
Java Result: 1
```

- What would you expect if you attempted to divide a floating point number by zero?

Programming Style & Documentation

- Three kinds of comments:
  - Line Comments: begin with `//`, extend to end of line.
  - Block Comments: begin with `/*`, end with `*/`.
  - Javadoc Comments: begin with `/**`, end with `*/`.
- Comment your code so it is understandable to another person.
- At a minimum, each method should have a brief comment at the top, explaining what the method does.

  (In Java a "method" is like a C++ "function".)

Reading Input from the Console

1. Create a Scanner object

```
Scanner input = new Scanner(System.in);
```

2. Use one of the class methods to obtain a value of the desired type. For example:

```
System.out.print("Enter a double value: ");
double d = input.nextDouble();

System.out.print("Enter an integer value: ");
double i = input.nextInt();
```

3. If the user enters text which does not comply with the requested type, a runtime exception will result. (We will talk more about this in an upcoming lecture.)

Summary

- Talk to me if you have not completed the primary course prerequisite: CSC-120.
- Do your best to keep up with reading the textbook.
- Object-Oriented Programming Concepts
- Online Documentation
- Make your programs understandable to another person.